Blackbox Testing Report

Test Cases

Addresses API

1. Read addresses list

- Input: GET `/addresses`
- Expected output: HTTP 200 and list response.
- Justification: This is the basic address read flow users depend on.
- Covers tests: `test_get_addresses`

2. Create address validation (valid + invalid)

- Input: POST `/addresses` with valid and invalid payloads (label, street, city, pincode length, empty pincode)
- Expected output: Valid payload succeeds (200/201). Invalid payloads fail (400).
- Justification: Bad address validation causes delivery failures and bad user data.
- Covers tests: `test_add_address_valid`, `test_add_address_invalid_label`, `test_add_address_invalid_street_short`, `test_add_address_invalid_city_short`, `test_add_address_invalid_pincode_length`, `test_add_address_empty_pincode_rejected`

3. Address update/default/delete behavior

- Input: PUT `/addresses/{id}` to set default/update fields, DELETE existing and non-existing ids
- Expected output: New default unsets old default, restricted fields are enforced, delete works, missing id returns 404.
- Justification: These are day-to-day address management actions users perform.
- Covers tests: `test_default_address_unsets_others`, `test_update_address_restrict_fields`, `test_delete_address`, `test_delete_non_existent_address`

4. Address pincode variant cases

- Input: POST `/addresses` with pincode variants `abcdef`, `12 456`, `12a456`, `1234-6`
- Expected output: HTTP 400 for each variant.
- Justification: Prevents malformed pin codes from entering the system.
- Covers tests: `test_add_address_invalid_pincode_format_variants[abcdef]`

Admin API

5. Admin list endpoints

- Input: GET `/admin/users`, `/admin/carts`, `/admin/orders`, `/admin/products`, `/admin/coupons`, `/admin/tickets`, `/admin/addresses`
- Expected output: HTTP 200 with list data.
- Justification: These endpoints power admin visibility and many test setups.
- Covers tests: `test_admin_get_users`, `test_admin_get_carts`, `test_admin_get_orders`, `test_admin_get_products`, `test_admin_get_coupons`, `test_admin_get_tickets`, `test_admin_get_addresses`

6. Admin user lookup and not found

- Input: GET `/admin/users/1`, GET `/admin/users/999999`
- Expected output: Existing id returns 200; missing id returns 404.
- Justification: User lookup behavior must be predictable for admin tools.
- Covers tests: `test_admin_get_user_by_id`, `test_admin_get_user_by_id_nonexistent_case`

7. Admin user id variant cases

- Input: GET `/admin/users/{id}` with `-1`, `abc`, `1.5`,
- Expected output: 400 or 404.
- Justification: Blocks malformed path ids from producing bad behavior.
- Covers tests: `test_admin_get_user_by_id_invalid_format_variants[-1]`

Auth / Header Validation

8. Required header checks

- Input: GET `/profile` with missing/invalid headers; GET `/admin/users` without `X-User-ID`
- Expected output: Missing/invalid user headers fail; admin endpoint without user id still works.
- Justification: Header checks are the first security and routing gate.
- Covers tests: `test_missing_roll_number`, `test_invalid_roll_number`, `test_missing_user_id`, `test_invalid_user_id`, `test_admin_missing_user_id`

9. Header variant cases

- Input: Invalid roll variants (`""`, `2024101104x`, `abc`) and invalid user id variants (`-1`, `abc`, `1.5`)
- Expected output: Roll variants return 400/401; user-id variants return 400.
- Justification: Prevents malformed headers from slipping through validation.
- Covers tests: `test_invalid_roll_number_variants[]`, `test_invalid_user_id_variants[-1]`

Cart API

10. Cart baseline and not found

- Input: GET `/cart`, add unknown product, remove product not in cart
- Expected output: GET works; unknown product/remove-not-present returns 404.
- Justification: This is baseline reliability for cart operations.
- Covers tests: `test_cart_clear_and_get`, `test_cart_add_product_not_found`, `test_cart_remove_product_not_in_cart`

11. Cart quantity + required-field validation

- Input: add/update with invalid quantities, add without quantity, update without product_id
- Expected output: HTTP 400 for invalid quantity/missing required fields.
- Justification: Stops invalid cart states before checkout.
- Covers tests: `test_cart_add_invalid_quantity_zero`, `test_cart_add_invalid_quantity_negative`, `test_cart_add_quantity_more_than_stock`, `test_cart_update_invalid_quantity`, `test_cart_add_requires_quantity`, `test_cart_update_requires_product_id`

12. Cart state and money calculations

- Input: add same product twice, large quantity subtotal, multi-item total comparison
- Expected output: quantities accumulate, subtotal is quantity×price, total equals sum of subtotals.
- Justification: This is the core billing correctness in cart.
- Covers tests: `test_cart_add_same_product_accumulates`, `test_cart_item_subtotal_exact_for_large_quantity`, `test_cart_subtotal_and_total_calculation`

13. Cart add payload variant cases

- Input: POST `/cart/add` with `{}`, `{"product_id":1}`, `{"quantity":1}`, `{"product_id":"one","quantity":1}`, `{"product_id":1,"quantity":"two"}`, `{"product_id":null,"quantity":1}`
- Expected output: handled safely with 200/400/404 depending on payload meaning.
- Justification: Ensures malformed request bodies do not crash the API.
- Covers tests: `test_cart_add_payload_variants_validation[payload0]`

14. Cart update payload variant cases

- Input: POST `/cart/update` with `{}`, `{"product_id":1}`, `{"quantity":1}`, `{"product_id":"x","quantity":2}`
- Expected output: handled safely with 200/400/404 depending on payload meaning.
- Justification: Update path should be as robust as add path.
- Covers tests: `test_cart_update_payload_variants_validation[payload0]`

Checkout API

15. Checkout method and state rules

- Input: invalid method, empty cart, CARD/COD/WALLET paths, COD over-limit case
- Expected output: invalid/empty fail; CARD is PAID; COD/WALLET expected pending; COD over 5000 rejected.
- Justification: Checkout is business-critical. Wrong states here break payments and orders.
- Covers tests: `test_checkout_rejects_invalid_payment_method`, `test_checkout_rejects_empty_cart`, `test_checkout_card_starts_paid`, `test_checkout_cod_starts_pending`, `test_checkout_wallet_starts_pending`, `test_checkout_cod_over_5000_rejected`

16. Checkout payment payload variant cases

- Input: `{}`, `{"payment_method":""}`, `{"payment_method":" "}`, `{"payment_method":"card"}`, `{"payment_method":"NETBANKING"}`, `{"payment_method":null}`, `{"payment_method":123}`
- Expected output: HTTP 400 for each.
- Justification: Prevents invalid payment methods from entering order flow.
- Covers tests: `test_checkout_payment_method_variants_validation[payload0]`

Coupon API

17. Coupon baseline + percent calculation

- Input: invalid code, expired code, remove without apply, apply PERCENT10 on known cart
- Expected output: invalid/expired fail, remove behaves gracefully, percent discount uses correct math.
- Justification: Coupon mistakes directly change final billed amount.
- Covers tests: `test_apply_coupon_invalid_code`, `test_apply_expired_coupon_rejected`, `test_remove_coupon_without_apply`, `test_apply_percent_coupon_calculation`

18. Coupon code variant cases

- Input: `""`, `" "`, `percent10`, `"PERCENT10 "`, `" PERCENT10"`, `@@@`, long 80-char code
- Expected output: safe response (200/400/404), never crash.
- Justification: Real users paste malformed codes; API should handle it safely.
- Covers tests: `test_apply_coupon_code_variants_do_not_500[]`

Loyalty API

19. Loyalty baseline + redeem boundaries

- Input: get loyalty, redeem 0, redeem negative, redeem over available points
- Expected output: GET succeeds; invalid/insufficient redeem returns 400.
- Justification: Users notice loyalty bugs quickly because points are value-like.
- Covers tests: `test_get_loyalty`, `test_loyalty_redeem_minimum_one`, `test_loyalty_redeem_negative_rejected`, `test_loyalty_redeem_insufficient_points`

20. Loyalty points-type variant cases

- Input: `points` as `"10"`, `"1.5"`, `null`, `" "`
- Expected output: HTTP 400.
- Justification: Prevents type mistakes in points redemption.
- Covers tests: `test_loyalty_redeem_points_type_variants_validation[ ]`

Orders API

21. Orders read and invoice consistency

- Input: place order, read list/detail, fetch invoice and compare totals
- Expected output: created order is visible and invoice math is consistent.
- Justification: Users depend on order history and invoice accuracy.
- Covers tests: `test_orders_list_and_get_by_id`, `test_invoice_has_consistent_totals`

22. Orders cancel behavior

- Input: cancel missing order, cancel placed order, try cancel delivered order
- Expected output: missing id returns 404; cancel should restore stock; delivered order cancel should be blocked.
- Justification: Cancellation errors cause stock mismatches and invalid order states.
- Covers tests: `test_cancel_missing_order_returns_404`, `test_cancel_order_restores_stock`, `test_cancel_delivered_order_rejected_if_present`

23. Orders id-format variant cases

- Input: bad ids (`abc`, `1.5`, , `-1`) across order detail/cancel/invoice endpoints
- Expected output: 400 or 404 for each.
- Justification: Hardens all order-id routes against malformed inputs.
- Covers tests: `test_order_endpoints_invalid_id_format_variants[ ]`

Products API

24. Product retrieval, filter, sort, and search behavior

- Input: active list check, invalid id, price parity user/admin, sort asc/desc, category filter, name search
- Expected output: inactive products hidden, invalid id 404, prices consistent, sort/filter/search semantics hold.
- Justification: Product browsing is high-traffic and user-visible; wrong data here is obvious.
- Covers tests: `test_get_products_only_active`, `test_get_product_by_invalid_id`, `test_product_price_matches_admin_data`, `test_products_sort_price_ascending`, `test_products_sort_price_descending`, `test_products_filter_by_category`, `test_products_search_by_name`

25. Product sort variant cases

- Input: sort values `""`, `PRICE_ASC`, `drop table`, `null`, `random`
- Expected output: 200 or 400, never 500.
- Justification: Invalid sort input should fail safely, not crash queries.
- Covers tests: `test_products_sort_variants_do_not_500[]`

26. Product search variant cases

- Input: search values `""`, `" "`, `a`, `A`, `@#$%`, very long string
- Expected output: 200 or 400, never 500.
- Justification: Search inputs are messy in real use. Stability matters.
- Covers tests: `test_products_search_variants_do_not_500[]`

Profile API

27. Profile read + valid update

- Input: GET `/profile`; PUT `/profile` with valid name/phone; GET again
- Expected output: read succeeds; update persists values.
- Justification: This is basic account-management behavior users expect to work.
- Covers tests: `test_get_profile`, `test_update_profile_valid`

28. Profile invalid name/phone core checks

- Input: short/long name, short/long phone, non-digit phone, whitespace-only name, phone with spaces
- Expected output: HTTP 400 for invalid updates.
- Justification: Prevents broken contact data from being stored.
- Covers tests: `test_update_profile_invalid_name_short`, `test_update_profile_invalid_name_long`, `test_update_profile_invalid_phone_short`, `test_update_profile_invalid_phone_long`, `test_update_profile_invalid_phone_chars`, `test_update_profile_whitespace_name_rejected`, `test_update_profile_phone_with_space_rejected`

29. Profile name variant cases

- Input: name variants `""`, `"A"`, `"A"*51` with valid phone
- Expected output: HTTP 400.
- Justification: Enforces name constraints consistently for edge cases.
- Covers tests: `test_update_profile_invalid_name_variants[]`

30. Profile phone variant cases

- Input: phone variants `""`, `"123"`, `"123456789012"` with valid name
- Expected output: HTTP 400.
- Justification: Blocks malformed phone values from profile records.
- Covers tests: `test_update_profile_invalid_phone_variants[]`

Reviews API

31. Reviews baseline and average behavior

- Input: get reviews, invalid rating/comment cases, valid reviews, average range and precision checks
- Expected output: shape is valid; invalid payloads fail; average in range and accurate for decimal case.
- Justification: Reviews directly influence buying decisions, so validation and math both matter.
- Covers tests: `test_reviews_get`, `test_reviews_reject_rating_out_of_range`, `test_reviews_reject_empty_comment`, `test_reviews_reject_comment_over_200`, `test_reviews_average_in_range`, `test_reviews_average_decimal_precision`

32. Reviews rating variant cases

- Input: rating variants `0`, `-1`, `6`, `"5"`, `"bad"`, `null`, `3.5`
- Expected output: HTTP 400.
- Justification: Prevents invalid rating values from corrupting aggregate ratings.
- Covers tests: `test_reviews_rating_type_and_range_variants_validation[0]`

Support API

33. Support lifecycle and transition rules

- Input: create ticket, valid transitions, invalid direct/rollback transitions, short subject, long message
- Expected output: valid flow succeeds; invalid transitions/fields fail with 400.
- Justification: Support workflow needs strict state rules to avoid operational confusion.
- Covers tests: `test_support_create_ticket_and_status_flow`, `test_support_reject_subject_too_short`, `test_support_reject_message_too_long`, `test_support_invalid_status_transition_rejected`, `test_support_reject_closed_to_in_progress`

34. Support ticket payload variant cases

- Input: `{}`, missing subject, missing message, wrong types, whitespace-only fields
- Expected output: HTTP 400.
- Justification: Stops malformed tickets from entering support queue.
- Covers tests: `test_support_create_ticket_payload_variants_validation[payload0]`

Wallet API

35. Wallet baseline + amount boundary checks

- Input: get balance, invalid add/pay amounts, exact add-pay arithmetic, insufficient-balance payment
- Expected output: get works; invalid values fail; add/pay math is exact; overpay fails.
- Justification: Wallet issues are money issues. These checks prevent financial mistakes.
- Covers tests: `test_get_wallet`, `test_wallet_add_invalid_amount_zero`, `test_wallet_add_invalid_amount_too_large`, `test_wallet_pay_invalid_amount_zero`, `test_wallet_pay_invalid_amount_negative`, `test_wallet_add_and_pay_exact_deduction`, `test_wallet_pay_insufficient_balance`

36. Wallet add amount-type variant cases

- Input: add amount as `"100"`, `"10.5"`, `null`, `" "`
- Expected output: HTTP 400.
- Justification: Prevents invalid types in wallet credit operations.
- Covers tests: `test_wallet_add_amount_type_variants_validation[100]`

37. Wallet pay amount-type variant cases

- Input: pay amount as `"50"`, `"1.25"`, `null`, `" "`
- Expected output: HTTP 400.
- Justification: Prevents invalid types in wallet debit operations.
- Covers tests: `test_wallet_pay_amount_type_variants_validation[50]`

Bug Report

Bug 1: Address API rejects valid 6-digit pincode

- Endpoint tested: `POST /api/v1/addresses`
- Request payload (with method, URL, headers, and body):
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/addresses`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

```
{
  "label": "HOME",
  "street": "123 Main Street",
  "city": "Metropolis",
  "pincode": "123456"
}
```

- Expected result (based on API doc): A valid 6-digit pincode should be accepted and the address should be created (HTTP 200/201).
- Actual result observed: API returns HTTP 400 and rejects this valid pincode.

Bug 2: Address API accepts invalid 5-digit pincode

- Endpoint tested: `POST /api/v1/addresses`
- Request payload (with method, URL, headers, and body):
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/addresses`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

```
{
  "label": "HOME",
  "street": "123 Main Street",
  "city": "Metropolis",
  "pincode": "12345"
}
```

- Expected result (based on API doc): A 5-digit pincode should be rejected with HTTP 400.
- Actual result observed: API accepts this payload and creates the address.

Bug 3: Cart allows quantity 0 during add

- Endpoint tested: `POST /api/v1/cart/add`
- Request payload (with method, URL, headers, and body):
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/cart/add`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

    ```
    {
      "product_id": 1,
      "quantity": 0
    }
    ```

- Expected result (based on API doc): Quantity must be at least 1, so this should return HTTP 400.
- Actual result observed: API accepts the payload and returns HTTP 200.

Bug 4: Cart total is incorrect

- Endpoint tested: `GET /api/v1/cart` (after cart setup)
- Request payload (with method, URL, headers, and body):
  - Step 1:
    - Method: `DELETE`
    - URL: `http://localhost:8080/api/v1/cart/clear`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
  - Step 2:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/cart/add`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
      - `Content-Type: application/json`
    - Body:

      ```
      {
        "product_id": 1,
        "quantity": 1
      }
      ```

  - Step 3:
    - Method: `GET`
    - URL: `http://localhost:8080/api/v1/cart`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
- Expected result (based on API doc): `total` should match the sum of item subtotals.
- Actual result observed: `total` is incorrect and appears as 0 even when item subtotal is positive.

Bug 5: Cart item subtotal overflows for larger quantities

- Endpoint tested: `GET /api/v1/cart` (after adding a large quantity)
- Request payload (with method, URL, headers, and body):
  - Step 1:
    - Method: `DELETE`
    - URL: `http://localhost:8080/api/v1/cart/clear`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
  - Step 2:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/cart/add`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
      - `Content-Type: application/json`
    - Body:

      ```
      {
        "product_id": 5,
        "quantity": 20
      }
      ```

  - Step 3:
    - Method: `GET`
    - URL: `http://localhost:8080/api/v1/cart`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
- Expected result (based on API doc): Subtotal should be `20 x 250 = 5000`.
- Actual result observed: Subtotal becomes wrapped or negative (example: `-120`).

Bug 6: COD checkout sets wrong payment status

- Endpoint tested: `POST /api/v1/checkout`
- Request payload (with method, URL, headers, and body):
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/checkout`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

    ```
    {
      "payment_method": "COD"
    }
    ```

- Expected result (based on API doc): COD orders should start with payment status `PENDING`.
- Actual result observed: API returns payment status as `PAID`.

Bug 7: Product price mismatch across APIs

- Endpoint tested: `GET /api/v1/products` and `GET /api/v1/admin/products`
- Request payload (with method, URL, headers, and body):
  - Request A:
    - Method: `GET`
    - URL: `http://localhost:8080/api/v1/products`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
  - Request B:
    - Method: `GET`
    - URL: `http://localhost:8080/api/v1/admin/products`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
- Expected result (based on API doc): The same product should show the same price in both endpoints.
- Actual result observed: At least one product shows different prices across the two endpoints.

Bug 8: Profile phone validation allows non-digit characters

- Endpoint tested: `PUT /api/v1/profile`
- Request payload (with method, URL, headers, and body):
  - Method: `PUT`
  - URL: `http://localhost:8080/api/v1/profile`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

    ```
    {
      "name": "Valid Name",
      "phone": "12345abcde"
    }
    ```

- Expected result (based on API doc): Phone must contain only digits and should be rejected with HTTP 400.
- Actual result observed: API accepts this invalid phone value and returns HTTP 200.

Bug 9: Support ticket status allows invalid transition

- Endpoint tested: `PUT /api/v1/support/tickets/{ticket_id}`
- Request payload (with method, URL, headers, and body):
  - Step 1 (create ticket):
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/support/tickets`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
      - `Content-Type: application/json`
    - Body:

      ```
      {
        "subject": "Test",
        "message": "Test ticket"
      }
      ```

  - Step 2 (invalid transition):
    - Method: `PUT`
    - URL: `http://localhost:8080/api/v1/support/tickets/{ticket_id}`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
      - `Content-Type: application/json`
    - Body:

      ```
      {
        "status": "CLOSED"
      }
      ```

- Expected result (based on API doc): Direct `OPEN` to `CLOSED` should be rejected with HTTP 400.
- Actual result observed: API accepts the transition and closes the ticket.

Bug 10: Wallet pay deducts incorrect amount

- Endpoint tested: `GET /api/v1/wallet`, `POST /api/v1/wallet/add`, `POST /api/v1/wallet/pay`
- Request payload (with method, URL, headers, and body):
  - Step 1:
    - Method: `GET`
    - URL: `http://localhost:8080/api/v1/wallet`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
  - Step 2:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/wallet/add`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
      - `Content-Type: application/json`
    - Body:
      ```
      {
        "amount": 100
      }
      ```
  - Step 3:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/wallet/pay`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
      - `Content-Type: application/json`
    - Body:
      ```
      {
        "amount": 40
      }
      ```
- Expected result (based on API doc): Final balance should increase by exactly 60.
- Actual result observed: Final balance is off by 0.40.

Bug 11: Percent coupon is calculated as flat discount

- Endpoint tested: `POST /api/v1/coupon/apply`
- Request payload (with method, URL, headers, and body):
  - Step 1:
    - Method: `DELETE`
    - URL: `http://localhost:8080/api/v1/cart/clear`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
  - Step 2:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/cart/add`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
      - `Content-Type: application/json`
    - Body:

      ```
      {
        "product_id": 1,
        "quantity": 3
      }
      ```

  - Step 3:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/coupon/apply`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
      - `Content-Type: application/json`
    - Body:

      ```
      {
        "coupon_code": "PERCENT10"
      }
      ```

- Expected result (based on API doc): For cart total 360, a 10 percent coupon should apply a discount of 36.
- Actual result observed: API applies a flat discount of 10.

Bug 12: Cancelling an order does not restore product stock

- Endpoint tested: `POST /api/v1/orders/{order_id}/cancel`
- Request payload (with method, URL, headers, and body):
  - Step 1:
    - Method: `GET`
    - URL: `http://localhost:8080/api/v1/admin/products`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
  - Step 2:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/checkout`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
      - `Content-Type: application/json`
    - Body:

      ```
      {
        "payment_method": "COD"
      }
      ```

  - Step 3:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/orders/{order_id}/cancel`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
- Expected result (based on API doc): Product stock should be restored to its original value after cancellation.
- Actual result observed: Stock stays reduced after cancellation.

Bug 13: Delivered orders can still be cancelled

- Endpoint tested: `POST /api/v1/orders/{order_id}/cancel`
- Request payload (with method, URL, headers, and body):
  - Step 1:
    - Method: `GET`
    - URL: `http://localhost:8080/api/v1/orders`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
  - Step 2:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/orders/{order_id}/cancel`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
    - Note: `{order_id}` is an order with status `DELIVERED`.
- Expected result (based on API doc): Delivered orders should not be cancellable and should return HTTP 400.
- Actual result observed: API accepts cancellation and returns success.

Bug 14: WALLET checkout sets wrong payment status

- Endpoint tested: `POST /api/v1/checkout`
- Request payload (with method, URL, headers, and body):
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/checkout`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

    ```
    {
      "payment_method": "WALLET"
    }
    ```

- Expected result (based on API doc): Wallet checkout should start with payment status `PENDING`.
- Actual result observed: API returns payment status as `PAID`.

Bug 15: Review average is rounded down instead of decimal

- Endpoint tested: `POST /api/v1/products/{product_id}/reviews` and `GET /api/v1/products/{product_id}/reviews`
- Request payload (with method, URL, headers, and body):
  - Step 1:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/products/{product_id}/reviews`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
      - `Content-Type: application/json`
    - Body:
      ```
      {
        "rating": 4,
        "comment": "Good product"
      }
      ```
  - Step 2:
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/products/{product_id}/reviews`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
      - `Content-Type: application/json`
    - Body:
      ```
      {
        "rating": 5,
        "comment": "Excellent"
      }
      ```
  - Step 3:
    - Method: `GET`
    - URL: `http://localhost:8080/api/v1/products/{product_id}/reviews`
    - Headers:
      - `X-Roll-Number: 2024101104`
      - `X-User-ID: 1`
    - Body: none
- Expected result (based on API doc): Average rating should be `4.5`.
- Actual result observed: API returns `4`.

Bug 16: Support status can move backward from CLOSED to IN_PROGRESS

- Endpoint tested: `PUT /api/v1/support/tickets/{ticket_id}`
- Request payload (with method, URL, headers, and body):

- Step 1:
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/support/tickets`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

```
{
  "subject": "Test",
  "message": "Test message"
}
```

- Step 2:
  - Method: `PUT`
  - URL: `http://localhost:8080/api/v1/support/tickets/{ticket_id}`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

```
{
  "status": "IN_PROGRESS"
}
```

- Step 3:
  - Method: `PUT`
  - URL: `http://localhost:8080/api/v1/support/tickets/{ticket_id}`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

```
{
  "status": "CLOSED"
}
```

- Step 4:

        - Method: `PUT`
        - URL: `http://localhost:8080/api/v1/support/tickets/{ticket_id}`
        - Headers:
            - `X-Roll-Number: 2024101104`
            - `X-User-ID: 1`
            - `Content-Type: application/json`
        - Body:
```
{
  "status": "IN_PROGRESS"
}
```

- Expected result (based on API doc): Once closed, the ticket should not move back to `IN_PROGRESS`.
- Actual result observed: API accepts backward transition and reopens the ticket.

Bug 17: Address API accepts empty pincode

- Endpoint tested: `POST /api/v1/addresses`
- Request payload (with method, URL, headers, and body):
    - Method: `POST`
    - URL: `http://localhost:8080/api/v1/addresses`
    - Headers:
        - `X-Roll-Number: 2024101104`
        - `X-User-ID: 1`
        - `Content-Type: application/json`
    - Body:
```
{
  "label": "HOME",
  "street": "123 Main Street",
  "city": "Metropolis",
  "pincode": ""
}
```

- Expected result (based on API doc): Empty pincode should be rejected with HTTP 400.
- Actual result observed: API accepts empty pincode and creates the address.

Bug 18: Profile accepts phone numbers containing spaces

- Endpoint tested: `PUT /api/v1/profile`
- Request payload (with method, URL, headers, and body):
  - Method: `PUT`
  - URL: `http://localhost:8080/api/v1/profile`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

    ```
    {
      "name": "Valid Name",
      "phone": "12345 7890"
    }
    ```

- Expected result (based on API doc): Phone should contain only digits and should be rejected with HTTP 400.
- Actual result observed: API accepts the phone number with a space.

Bug 19: Cart add accepts payload without quantity

- Endpoint tested: `POST /api/v1/cart/add`
- Request payload (with method, URL, headers, and body):
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/cart/add`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

    ```
    {
      "product_id": 1
    }
    ```

- Expected result (based on API doc): Missing required field `quantity` should return HTTP 400.
- Actual result observed: API accepts this payload and returns HTTP 200.

Bug 20: Cart update accepts payload without product_id

- Endpoint tested: `POST /api/v1/cart/update`
- Request payload (with method, URL, headers, and body):
  - Method: `POST`
  - URL: `http://localhost:8080/api/v1/cart/update`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

    ```
    {
      "quantity": 1
    }
    ```

- Expected result (based on API doc): Missing required field `product_id` should return HTTP 400.
- Actual result observed: API accepts this payload and returns HTTP 200.

Bug 21: Profile accepts whitespace-only name

- Endpoint tested: `PUT /api/v1/profile`
- Request payload (with method, URL, headers, and body):
  - Method: `PUT`
  - URL: `http://localhost:8080/api/v1/profile`
  - Headers:
    - `X-Roll-Number: 2024101104`
    - `X-User-ID: 1`
    - `Content-Type: application/json`
  - Body:

    ```
    {
      "name": "  ",
      "phone": "1234567890"
    }
    ```

- Expected result (based on API doc): Name should not be only whitespace and should be rejected with HTTP 400.
- Actual result observed: API accepts the payload and returns HTTP 200.

Note

- One test is skipped because the delivered-order cancellation case depends on whether user 1 currently has a delivered order in the dataset.